Investigating Circuit Routing Protocols in Distributed Quantum Computing

SQA 2026 Undergraduate Research Project 16

Elan Blaustein, Sibishan Ravindran, Thien Phu Tran

March 1, 2026

Abstract

Scaling quantum computing beyond the Noisy Intermediate-Scale Quantum (NISQ) era requires distributed architectures (DQC) where multiple cores are connected via quantum interconnects. This report investigates the adaptation of the SWAP-based BidiRectional heuristic search (SABRE) algorithm for DQC. We analyse TeleSABRE, a modified heuristic cost function that accounts for inter-core communication penalties and teleportation costs. We also propose an alternative architecture model (BLaRT) to gauge algorithmic optimisation to improve on TeleSABRE's routing protocol efficiency. We evaluate our BLaRTSABRE implementation against baseline approaches using quantum circuit benchmarks, analysing the impact on circuit depth and SWAP overhead.

1 Introduction

Quantum circuit mapping, the process of transforming a logical quantum circuit to adhere to the connectivity constraints of a physical device, is a fundamental challenge in the compilation pipeline. For standard monolithic architectures, heuristic algorithms such as SABRE (SWAP-based BidiRectional heuristic search) have become the industry standard for minimising the number of added SWAP gates and reducing circuit depth [1]. Recently, enhanced versions like LightSABRE have further optimised these heuristics for scalability and backtracking capabilities [2].

However, as quantum processors scale, the process of manufacturing large, high-fidelity monolithic chips becomes increasingly difficult. Distributed Quantum Computing (DQC) addresses this scalability bottleneck by interconnecting smaller, modular quantum cores. In this architecture, communication between cores is achieved not through direct coupling, but via quantum teleportation protocols that consume pre-generated entanglement Bell states [3].

This architectural shift fundamentally alters the mapping problem. Unlike the relatively uniform cost of performing SWAP gates on a single chip, DQC architectures impose a greater penalty on inter-core operations. A standard mapping algorithm that treats qubits between distinct cores uniformly may produce a valid circuit, but it will likely incur prohibitive latency and resource costs by overusing the inter-core links.

In this report, we investigate the application of the SABRE algorithm to distributed architectures, building upon recent distributed extensions such as TeleSABRE [4]. We propose a modified heuristic operating on a novel architecture that carefully accounts for remote operations, localising sub-circuits within cores and eliminating concerns involving Bell state management.

2 Background: The SABRE Algorithm

2.1 The Qubit Mapping Problem

All non-trivial circuits involving entanglement require gates that operate on two or more qubits. Due to the existence of the universal quantum gate set [5], it is a given that to perform quantum algorithms on a physical computer, it is necessary to have a physical architecture permitting the execution of two-qubit gates.

The qubit connectivity of a quantum processor is represented by a coupling graph $G = (P, E)$, where P is the set of physical qubits and E represents the available two-qubit interactions via edges between qubits. One example of such is in Figure 1, depicting the coupling configuration of IBM's Q Tokyo device.

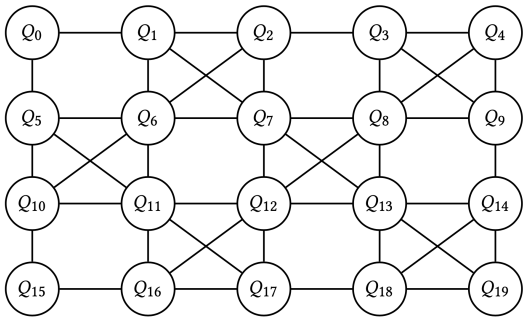


Figure 1: IBM Q Tokyo Architecture Coupling Graph [6]

A logical quantum circuit consists of a sequence of gates acting on a set of logical qubits L. To perform a circuit on physical hardware, a mapping $\pi : L \to P$ is required. This mapping appears simple to generate, assuming the physical architecture is large enough to support the logical circuit requirements.

However, a primary concern is that most NISQ devices do not support all-to-all connectivity. If a two-qubit gate acts on logical qubits l_1, l_2 mapped to physical qubits $\pi(l_1), \pi(l_2)$ which are not adjacent in G, the circuit cannot be executed directly. Even with an ideal initial mapping, it is incredibly unlikely that all two-qubit gates are satisfiable within the coupling graph. Therefore, the quantum circuit compiler must insert SWAP gates to permute the qubits during execution until the adjacency constraint is met. The goal of the mapping problem is to find an initial assignment π_0 and a schedule of SWAP gates that minimises the total circuit depth and gate count [1].

2.2 SABRE Overview

SABRE (SWAP-based BidiRectional heuristic search) is a state-of-the-art heuristic algorithm designed to address the scalability issues of exact mapping approaches. It operates in two key phases to optimize the logical-to-physical mapping:

1. **Heuristic Search (Routing):** For a fixed initial mapping, SABRE traverses the circuit's gate dependency graph as a Directed Acyclic Graph. When a gate cannot be executed, it generates a candidate set of SWAP operations (consisting of all edges incident to qubits involved in gates to be executed immediately) and selects the one that minimises a heuristic cost function. This process repeats until all the gates in the circuit have executed.

2. **Bidirectional Pass:** To optimise the *initial* mapping, SABRE performs iterative forward and backward passes. The final mapping of the forward pass is used as the initial mapping for the backward pass (routing the reversed circuit). This process effectively "anneals" the mapping towards a configuration that reduces global SWAP overhead [1].

2.3 Heuristic Cost Function

The core of SABRE is its scoring mechanism for selecting SWAP gates. The algorithm maintains a "Front Layer" ($\mathcal{F}$) of two-qubit executable gates and an "Extended Set" ($\mathcal{E}$) of future gates to provide lookahead capabilities.

For a candidate SWAP σ, the heuristic score $H(\sigma)$ is calculated as:

$$H(\sigma) = \delta(\sigma) \left(\frac{1}{|\mathcal{F}|} \sum_{g \in \mathcal{F}} D(\pi_\sigma(g[l_1]), \pi_\sigma(g[l_2])) + W \cdot \left(\frac{1}{|\mathcal{E}|} \sum_{g \in \mathcal{E}} D(\pi_\sigma(g[l_1]), \pi_\sigma(g[l_2])) \right) \right) \quad (1)$$

Where:

- $g[l_1]$ is the first logical qubit that gate g operates on (resp. l_2, second).
- $D(p_i, p_j)$ is the shortest-path distance between physical qubits p_i and p_j on the coupling graph G; this may be pre-computed using the Floyd-Warshall algorithm on G.
- π_σ is the mapping resulting from applying the SWAP σ to the current mapping π.
- $0 \leq W \leq 1$ is a weighting parameter (typically 0.5) that balances the immediate gain against future connectivity requirements.
- $\delta(\sigma)$ is a decay parameter that increases (typically by 0.01) every time a qubit is used in a SWAP operation; it is tuned so that a particular qubit is less likely to be swapped multiple times in a row (though it resets after approximately 5 iterations of the routing search).

Standard SABRE assumes a uniform distance metric where every edge in G has a weight of 1. In the following sections, we discuss how this assumption fails in Distributed Quantum Computing (DQC) architectures and introduce our modifications to the cost function.

With this heuristic, SABRE is able to successfully map and route quantum circuits of great complexity on a single device.

3 DQC Architecture Model

To address the limitations of monolithic scaling, we adopt and investigate the Distributed Quantum Computing (DQC) model, which provides better architectural flexibility and processing power for running quantum algorithm. This architecture networks multiple quantum processors (cores) to function as a single logical device using specialised quantum communication operations to transmit inter-core data.

3.1 Graph Representation

We model the distributed system as a coupling graph $G_{DQC} = (V, E_{local}, E_{comm})$, where V represents the set of physical qubits. E_{local} and E_{comm} are two disjoint sets of edges that collectively represent the available connectivity of the architecture. The overall edge set E is the union of the two sets:

$$E = E_{local} \cup E_{comm} \tag{2}$$

where E_{local} contains edges connecting qubits within the same core, and E_{tele} contains edges connecting qubits located in different cores. The system is partitioned into K distinct cores (or modules) $\{C_1, C_2, \ldots, C_K\}$, such that every physical qubit belongs to exactly one core. Figure 2 demonstrates the physical coupling layout of the one of the two architectures used in our investigation.

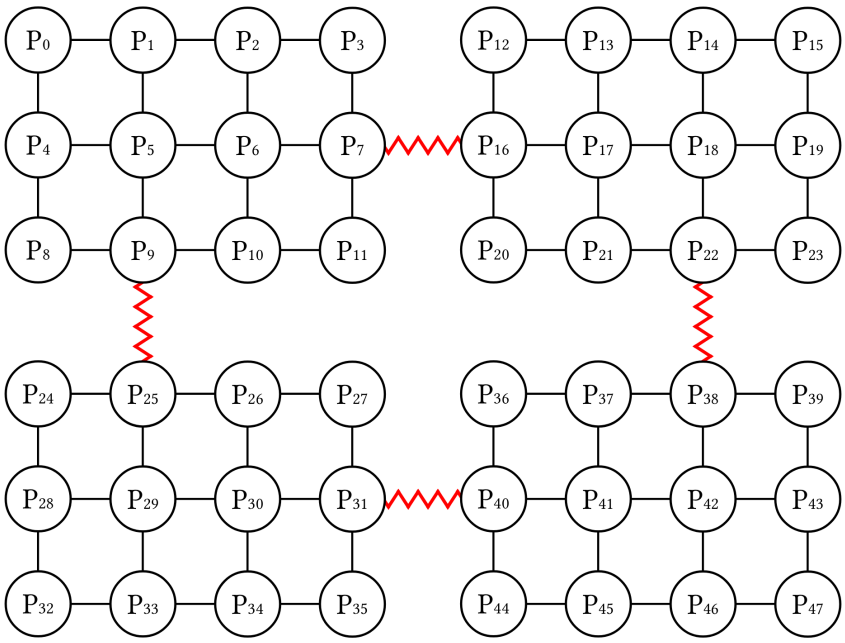


Figure 2: "Small" DQC Architecture Coupling Graph

The E_{local} edges behave exactly as described as with the monolithic architecture, but the E_{comm} edges are specialised in use to perform particular communication operations between the cores. Edges in E_{comm} are designated as "communication edges", and the qubits they connect to as "communication qubits".

3.2 Tele-Operations

In order to facilitate the transfer of information between cores, communication edges take advantage of quantum teleportation. By using just an entangled pair of qubits and classical bit communication, a communication edge is able to transfer the state of one qubit into another in a different core, or even to execute a two-qubit gate between two qubits in different cores. These kinds of operations are referred to as Tele-operations, in comparison to the Local SWAP operations that manipulate qubits within a core.

The two Tele-operations function using quantum teleportation principles as described in [7]; this process can be broken into two stages, cat-entanglement and cat-detanglement. The first step is cat-entanglement, which involves taking an input state $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$ and entangling it with a qubit in another core. It starts with the preparation of a Bell state $|\Phi^+\rangle$, which is the only direct quantum operation that occurs between the two cores in the entire process (Fig. 3).

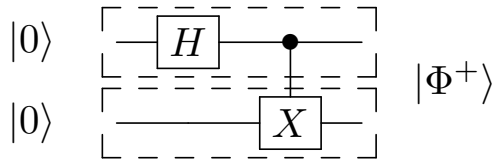


Figure 3: Bell State Generation Circuit

Once the Bell state has been distributed between the two cores, the entanglement circuit (Fig. 4) is executed, which distributes the $\alpha\,|0\rangle + \beta\,|1\rangle$ state across two qubits. One of the Bell state qubits is entangled with the source qubit and then measured, with the measurement being communicated between the cores as a control bit. Once this is performed, the source qubit and the remaining Bell state qubit will jointly form the state $\alpha\,|00\rangle + \beta\,|11\rangle$. This completes the cat-entanglement, and this final state is described as a "cat-like" state.

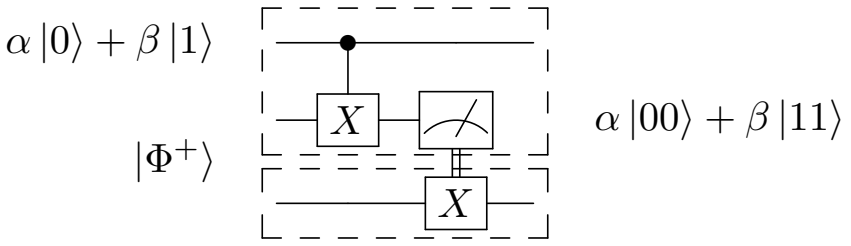


Figure 4: Cat-Entanglement Circuit

Once $|\psi\rangle$ has been put into a cat-like state, then the cat-detanglement circuit is executed. Since the cat-like state is symmetric, the target qubit of the cat-detanglement determines which Tele-operation is performed.

To move the state $|\psi\rangle$ into the target core, a Teleportation is performed (Fig. 5), where the source qubit is measured and communicated as a control bit. This undoes the cat-like state, causing the final $|\psi\rangle$ state to reside in the Bell state qubit of the opposite core, completing the teleportation of quantum information between cores.

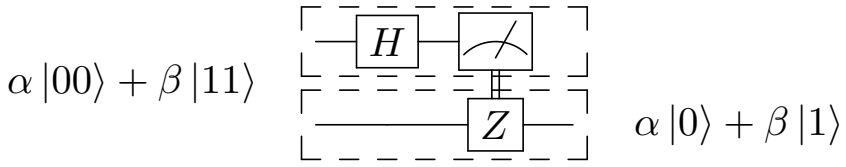


Figure 5: Cat-Detanglement Teleportation Circuit

If one were to perform the cat-detanglement circuit targeting the opposite qubit, then the $|\psi\rangle$ state would return to the original qubit, effectively doing nothing. However, as the cat-like state symmetrically represents the $|\psi\rangle$ state, then either qubit may be used as a control qubit prior to the cat-detanglement. By performing a controlled-gate operation to another qubit $|\chi\rangle$ in the opposite core, it is possible to keep $|\psi\rangle$ in its original core, yet involve it in a two-qubit operation in the target core; this operation is called a "Telegate".

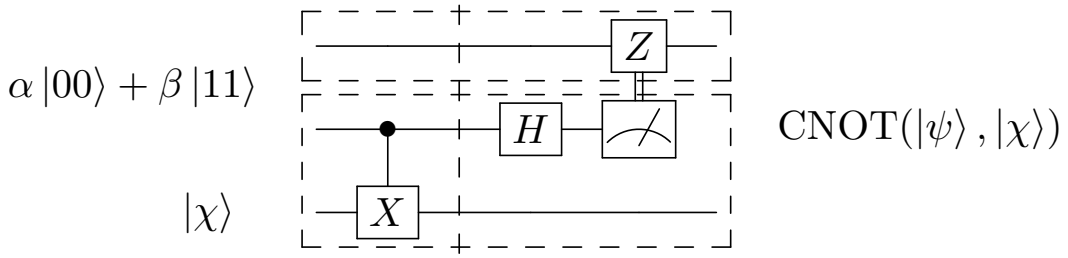


Figure 6: Cat-Detanglement Telegate Circuit

By allowing the quantum circuit compiler to perform Teleportations and Telegates alongside Local SWAPs, any circuit can now be run on DQC architecture, with one major consideration. Note that a Tele-operation requires the Bell state to be distributed among both cores; this effectively means that the two qubits connected by the communication edge cannot be used in the circuit (they are not mapped to by a logical qubit in π). These qubits are deemed "free qubits"; they can be moved around cores like any other qubit, but a Tele-operation can only be performed if both communication qubits it connects are free qubits, with $|\psi\rangle$ adjacent to the free qubit in the source core (additionally, a Telegate also requires $|\chi\rangle$ to be adjacent to the free qubit in the target core). The problem that arises is that the number of free qubits in each core is not invariant. When a Teleportation is performed, the $|\psi\rangle$ state is moved into the free qubit of the target core, causing the target core's free qubit to move to the original, source qubit. This effectively SWAPs 1 free qubit from the target core to the source core. If the numbers of free qubits are not managed properly, then it is possible for a core to lose all of its free qubits, preventing it from being able to perform any Tele-operations, halting the circuit's execution. This is termed a "deadlock". A proper DQC routing protocol must ensure that every core has enough free qubits for the entire execution of the quantum circuit.

4 The DQC-SABRE Implementation

In this section, we present **TeleSABRE**, a DQC-aware extension of the SABRE algorithm [4]. In order to account for the differences in the DQC model, this implementation uses an overhauled heuristic function, utilising a dynamic "Contracted Graph" model to evaluate the cost of inter-core operations, taking into account the higher overhead of Tele-operations, free qubit management, and deadlock prevention.

4.1 Dual-Candidate Heuristic Search

Standard SABRE only considers Local SWAP operations on directly-required qubits as candidate operations during the routing phase. TeleSABRE expands this search space by distinguishing the three types of movement candidates:

- **Local SWAPs:** Standard exchanges between adjacent physical qubits within a core. The algorithm considers a SWAP as a candidate if it SWAPs a qubit involved in a gate in $\mathcal{F}$, or if it moves a free qubit to a new location.
- **Teleportations:** Specific operations that move a logical qubit to a "communication qubit" to facilitate a transfer to another core. The algorithm considers all possible Teleportations where the following criteria is met:
  - A gate in $\mathcal{F}$ has two qubits in different cores
  - One of the gate's qubits is adjacent to a communication qubit that has an incident communication edge
  - The communication qubit and the target communication qubit neighbour are both free qubits
  - The target core has more than 1 free qubits

  Note that the Teleportation need not be to the nearest core to the gate's target qubit; the heuristic function is designed to choose the best Teleportation given the context of the entire circuit.
- **Telegates:** Specific operations that allow a two-qubit gate to be performed over a communication edge. The algorithm considers all possible Telegates where the following criteria is met:
  - A gate in $\mathcal{F}$ has two qubits in different cores
  - Both of the gate's qubits are adjacent to *connected* communication qubits
  - Both communication qubits are free qubits

For every routing step, the algorithm generates all sets of candidates and selects the operation that minimises the total heuristic energy H when applied to π.

4.2 Dynamic Heuristic Energy Evaluation via Contracted Graphs

The clever step used by the TeleSABRE algorithm to accurately estimate the cost of moving qubits between cores is through the implementation of a `contracted_graph` routine. This function constructs a temporary routing graph $G_{contract}$ that accounts for the availability of communication qubits, free qubits, the source/target qubits, and the distances between them. To calculate the distance between a gate's two qubits, the heuristic function executes `contracted_graph` to estimate how 'close' the qubits are, where a lower distance makes the gate more likely to be executable.

The process of constructing this contracted graph is illustrated in Figure 7. Given the mapping of a gate's logical qubits onto the physical architecture $\pi_\sigma(g)$, all edges in the coupling graph G are contracted, leaving only the nodes corresponding to the communication qubits and $\pi_\sigma(g)$. Compared to the monolithic model where all edge weights are equal, the contracted graph increases the weight of communication edges to 2 to indicate the higher overhead, while not completely preventing Tele-operations from being performed. After this, a free qubit penalty is applied to each communication qubit, calculated as the distance to the nearest free qubit within the communication qubit's core. This ensures that candidate operations that place free qubits nearer to communication qubits have better scores. Additionally, a full core penalty of 10 is applied to all qubits located in a core with at most 2 free qubits (a "full core"). While performing a Tele-operation targeting a full core is permissible (and necessary in some cases), it should be minimised where possible to prevent the occurrence of deadlocks. Finally, once $G_{contract}(\pi_\sigma(g))$ has been completely constructed, Dijkstra's algorithm is used to find the shortest distance between the two $\pi_\sigma(g)$ qubits, concluding the distance calculation subroutine.

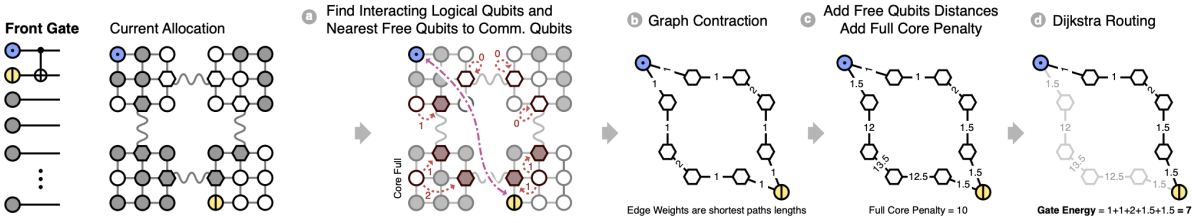


Figure 7: Heuristic Graph Contraction [4]

The cost function for a candidate operation is nearly identical to that of SABRE's, differing only in the distance calculation.

$$H(\sigma) = \delta(\sigma)\left(\frac{1}{|\mathcal{F}|}\sum_{g\in\mathcal{F}} D(\pi_\sigma(g)) + W \cdot \left(\frac{1}{|\mathcal{E}|}\sum_{g\in\mathcal{E}} D(\pi_\sigma(g))\right)\right) \tag{3}$$

Where the distance function D calls the `contracted_graph` routine (but only if the physical qubits g maps to are in different cores).

$$D(\pi_\sigma(g)) = \begin{cases} Dist_G(\pi_\sigma(g)[1], \pi_\sigma(g)[2]) & \text{if } \pi_\sigma(g)[1] \text{ and } \pi_\sigma(g)[2] \text{ are in the same core} \\ Dist_{G_{contract}(\pi_\sigma(g))}(\pi_\sigma(g)[1], \pi_\sigma(g)[2]) & \text{otherwise} \end{cases} \tag{4}$$

Additionally, W is set to a smaller value than in ordinary SABRE (around 0.01), since the algorithm must prioritise current gates in $\mathcal{F}$ to prevent deadlocks.

4.3 Algorithmic Optimisations

The above section covers all the novel concepts that the TeleSABRE paper introduces, however there are various key optimisation steps that must be made in order for the algorithm to run practically. After investigating the TeleSABRE repository [8], we have identified all additional elements required for the algorithm to run efficiently and avoid deadlocks. Multiple optimisations are approaches to mitigate a recurring problem referred to as the "ping-pong" effect. This occurs when a qubit is swapped back and forth, either between two cores or within one core, to satisfy alternating, opposing gate dependencies with equal gate heuristic scores. This is the most common cause for the algorithm to fail, and as such, these optimisations are necessary to minimise its appearances as much as possible.

- **Deadlock Timer:** A 'deadlock_timer' (set to 80 steps) monitors progress, counting down every time the heuristic energy calculation is run for movement candidates. If no circuit gates are executed within this window, the pass is flagged as a potential deadlock.

- **Reset Timer:** When the pass is flagged as a potential deadlock after the reset timer counts down, then the algorithm goes into "deadlock solving" mode. This mode prevents the "ping-pong" effect by removing the cause: conflicting dependencies in the heuristic calculation. When determining the heuristic value, only a single gate in the front layer is used for calculations: $H(\sigma) = \delta(\sigma)\left(D(\pi_\sigma(\mathcal{F}_0))\right)$. This essentially forces the algorithm to focus on executing that one gate until is is successfully executed, where it returns to normal operation. Additionally, the algorithm resets the state of the mapping to where it was right after the last gate was successfully executed to cut down on useless operations. If the algorithm iterates for another 150 steps with no improvement, the pass is marked as a fail and halts.

- **Random Best Operation:** Since the "ping-pong" effect is characterised by picking the same operation repeatedly, even when other operations may be scored equally, it may be necessary to add some randomness to the choice of the best operation to execute. Therefore, when choosing from multiple operations all tied for the minimal heuristic score, the algorithm should choose one at random.

- **Communication Qubit Contraction Penalty:** During the `contracted_graph` routine, a part of the distance calculation metric involves the distance from a gate's source/target qubits to the nearest communication qubit. However, this is not optimal for performing Tele-operations, since the heuristic scores a mapping better the closer the source/target qubits are to the communication qubit, with the best score for placing the source/target qubits *in* the communication qubit. However, since Tele-operations require free communication qubits in order to execute, this should be discouraged. As such, an additional penalty is added to any communication node that contains the source/target qubit.

- **Traffic:** Whenever a Tele-operation is performed over a communication, it's not always preferable for another Tele-operation to be performed immediately after over the same communication edge, in case the free qubits are not well-balanced between the cores. To indicate this to the heuristic function, a `traffic` data structure keeps track of all Dijkstra-routed paths and records every time a communication edge is traversed. When the next gate in $\mathcal{F}$ is undergoing the `contracted_graph` routine, `traffic` adds a corresponding weight penalty to every traversed edge, making it less likely for the next Dijkstra routing to re-use the same edge.

- **Deadlock Teleports:** In situations where the algorithm is close to deadlocking (e.g. multiple cores only have 1 free qubit remaining), then it may be necessary to add Teleportation operations that serve the purpose of redistributing free qubits, rather than moving around source/target qubits. Any Teleportation that teleports a free qubit into a core with very few qubits is considered as an additional Teleportation candidate.

- **Initial Mapping Optimisation:** While TeleSABRE works for any given random mapping π (with no deadlocked cores), this mapping can be chosen to optimise the beginning of the algorithm. By iterating through two-qubit gates in $\mathcal{F}$, qubit pairs can be randomly assigned to cores such that most initial two-qubit gates can be ensured to be executable within a single core, saving Tele-operation costs.

- **Tele Bonus:** Since inter-core operations tend to have larger distance scores, the heuristic algorithm is biased towards Local SWAPs. While saving on overhead, this can prevent necessary Tele-operations from being executed and cause the "ping-pong" effect. As such, any Tele-operation is given a large score bonus of around -100 to prioritise them whenever they can be performed (not including deadlock Teleports).

- **Iterations:** Due to the stochastic nature of the algorithm, the resultant mapping and number of executed operations can vary every time the algorithm is run. To account for this, the algorithm can be run for a few iterations in order to pick best execution.

Algorithm 1 TeleSABRE Forward Pass [4]

Input: Circuit DAG, Architecture G_{DQC}
while FrontLayer is not empty **do**
    $execute\_gate\_list \leftarrow [\,]$
    $gate\_execution\_log \leftarrow [\,]$
    $decay \leftarrow$ OnesArray($G_{DQC}.num\_qubits$)
    for *gate in* FrontLayer **do**
        if $G_{DQC}.is\_gate\_executable(\pi(gate))$ **then**
            $execute\_gate\_list.append(gate)$
        end if
    end for
    if $execute\_gate\_list = [\,]$ **then**
        for *gate in execute_gate_list* **do**
            DAG.$remove(gate)$
            $gate\_execution\_log.append(gate)$
        end for
        $solving\_deadlock \leftarrow False$
        $solving\_deadlock\_timer \leftarrow 80$
        $reset\_timer \leftarrow 150$
    else
        Candidates $C \leftarrow$ GetSwaps($\mathcal{F}$)
        $score \leftarrow [\,]$
        for σ in C **do**
            $\pi_\sigma \leftarrow$ApplyOperation(π, σ)
            $score[\sigma] \leftarrow$ CalculateScore($\sigma, \mathcal{F}, \mathcal{E}, decay, solving\_deadlock$)
        end for
        $best\_\sigma \leftarrow \min(score)$
        $\pi \leftarrow$ApplyOperation($\pi, best\_\sigma$)
        $gate\_execution\_log.append(best\_\sigma)$
        $decay \leftarrow$ UpdateDecay($decay, best\_\sigma$)
    end if
    Update DAG, $\mathcal{F}, \mathcal{E}$
    UpdateTimers($solving\_deadlock, solving\_deadlock\_timer, reset\_timer$)
end while
return $\pi, gate\_execution\_log$

5 BLaRTSABRE

After investigating TeleSABRE, its use cases, how it performs, and the required optimisations, we have developed our own DQC paradigm that addresses the issues affecting TeleSABRE. We have formulated a novel protocol: the Bell-Linking and Remote Teleportation (BLaRT) system. This is comprised of a modification to typical DQC architectures, termed "BLaRT architecture", and a hybrid algorithm combining features of SABRE and TeleSABRE, "BLaRTSABRE".

5.1 The BLaRT Architecture

The motivation behind BLaRT comes from mitigating the complications found with ordinary DQC architectures, notably free qubit management and deadlocks. Repeatedly calculating contracted graphs and taking into account the distribution of free qubits introduces too much complexity into the problem; this is what BLaRT architecture does away with.

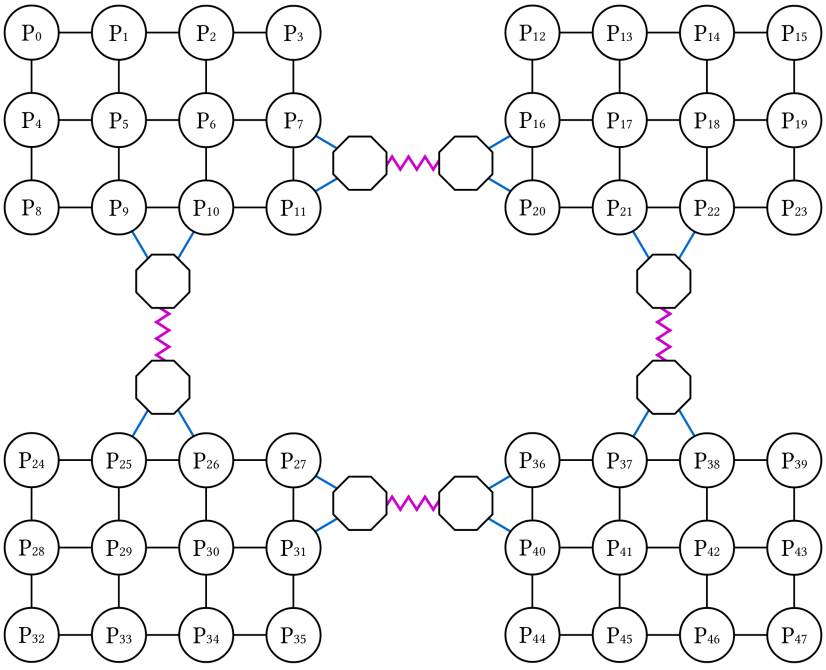


Figure 8: "Small" BLaRT Architecture Coupling Graph

All that BLaRT architecture does is that it replaces communication edges and communication qubits with "BLaRT edges" and "BLaRT nodes". Figure 8 shows the BLaRT equivalent to the DQC coupling graph in Figure 2. BLaRT edges perform all the handling of Bell states, free qubits, and inter-core operations in order to simplify the routing protocol itself.

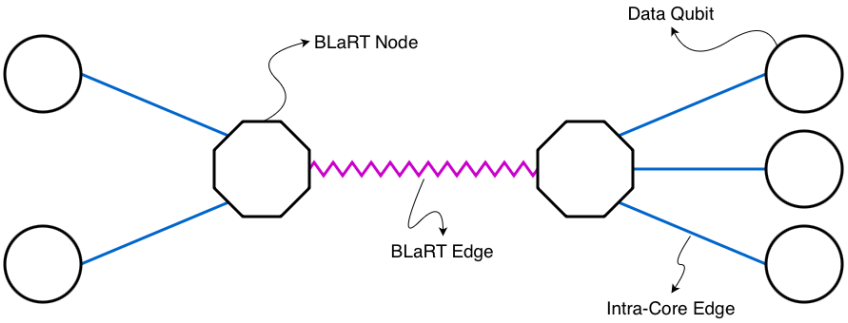


Figure 9: BLaRT Edge Node Representation

The representation of the BLaRT edge in Figure 9 is an abstraction of the actual components of a BLaRT edge, which are properly represented in Figure 10. Each BLaRT node is actually comprised of a pair of communication qubits, and each BLaRT edge is a pair of communication edges. When a qubit is coupled with a BLaRT node, it is actually coupled to both communication qubits in its core. Additionally, each communication edge is supported with an auxiliary Bell-Linking edge, each consisting of two minor qubits. Bell-Linking edges are dedicated to creating entangled Bell state qubit pairs pre-loaded for the communication edges. This reduces overhead, as Bell-linking can be done while inter-core operations are being executed, while also isolating all direct quantum interactions between cores, leaving the communication edges as purely classical communication channels.

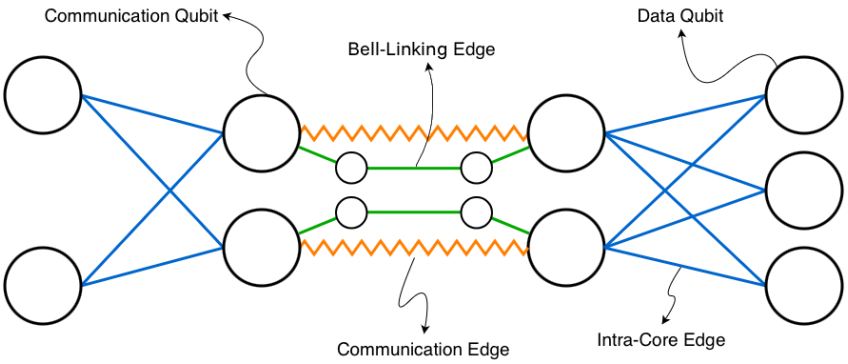


Figure 10: BLaRT Edge Underlying Communication Topology

With BLaRT edges, Tele-operations are supplanted by Remote Teleportations, inter-core data transfer operations that are not restricted by Tele-operation criteria in order to execute. These consist of Remote SWAPs and Remote Gates, replacing Teleportations and Telegates, respectively. BLaRT edges capable of performing Remote Teleportations on a simplified coupling graph can be represented as a bipartite graph, as a BLaRT edge allows for any connected qubit to interface with any connected qubit in the other core (Fig. 11). These edge abstractions are analogous to intra-core coupling edges, as both facilitate SWAPs of qubits and the execution of two-qubit gates.

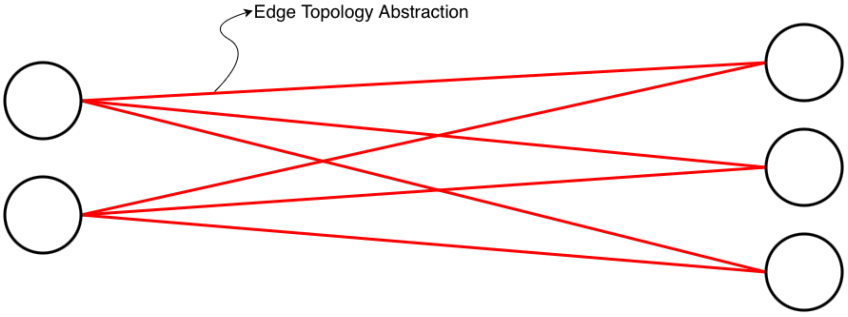
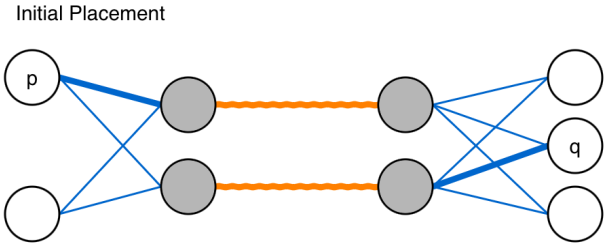


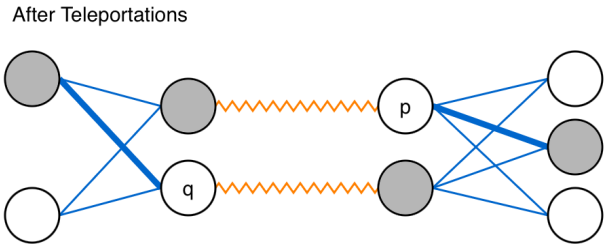
Figure 11: BLaRT Edge Virtual Topology

5.2 Remote Teleportations

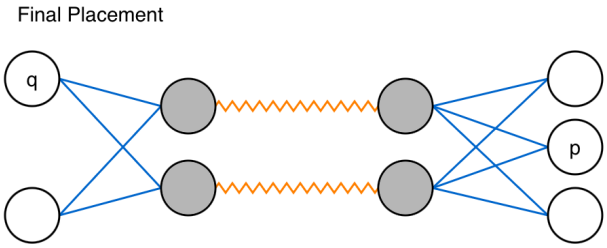
Remote SWAPs are, in practice, two simultaneous Teleportations. Instead of teleporting a single qubit from one core to another, which moves around free qubits, a double Teleportation maintains the free qubits it depends on, ensuring they remain in the communication qubits at all times.



(a) Remote SWAP State 1



(b) Remote SWAP State 2



(c) Remote SWAP State 3

Figure 12: Remote SWAP Steps

Figure 13 depicts the quantum circuit of a Remote SWAP, where the Bell-Linking edges have pre-loaded a Bell pair into the communication qubits.

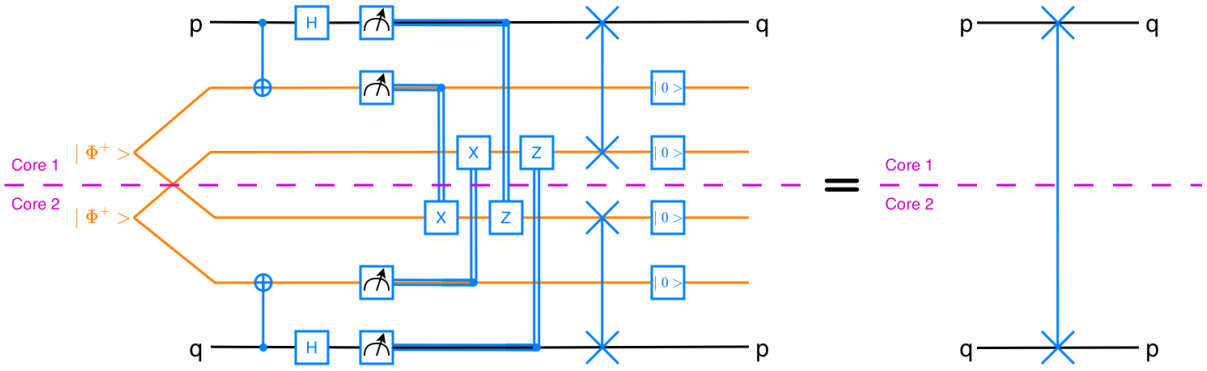


Figure 13: Remote SWAP Circuit

Remote Gates are essentially identical to Telegates, since an ordinary Telegate operation doesn't move qubits around. However, one minor upgrade is that Remote Gates can perform two Telegate operations on any two pairs of adjacent qubits.

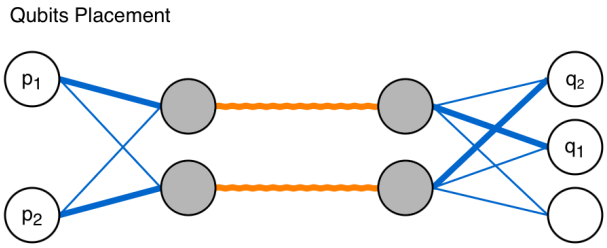


Figure 14: Remote Gate State

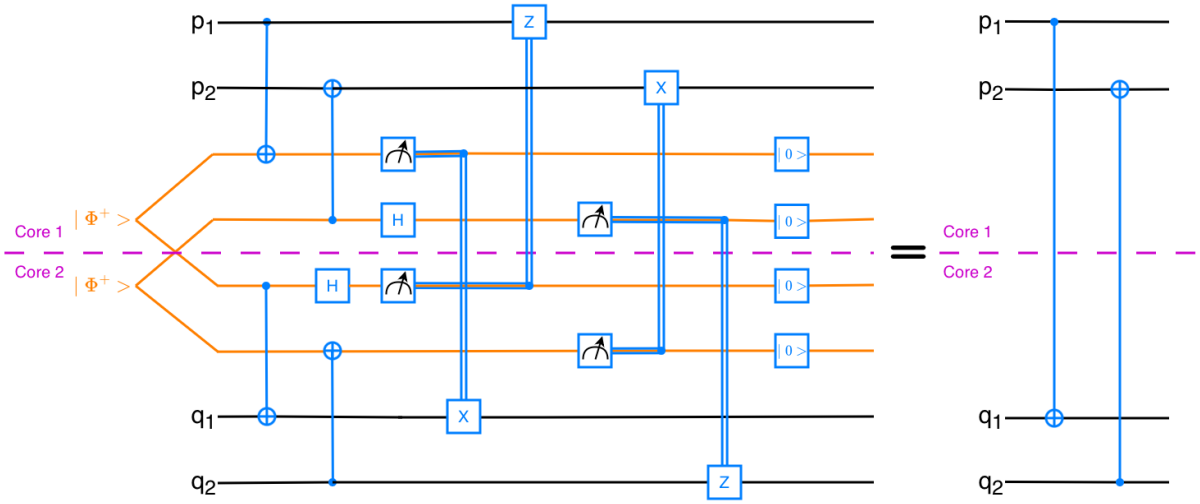


Figure 15: Remote Gate Circuit

Remote Teleportation operations effectively allow the entire BLaRT architecture to act as a monolithic device, despite its modular core composition. In addition, due to the parallel nature of these remote operations and the pre-loading of Bell states from the Bell-Linking edge, Remote SWAPs can be performed quicker than ordinary Teleportations, despite the apparent double cost. Likewise, a Remote Gate can be performed with a lower depth than a Telegate (not even taking into account that each BLaRT edge can facilitate two simultaneous Remote Gates simultanously). As such, the BLaRTSABRE routing protocol is able to take advantage of the efficiency of SABRE, while building upon and improving the optimisations present in TeleSABRE. No graph contraction, free qubit distance calculations, or Tele-operation candidate criteria are required for BLaRTSABRE.

5.3 BLaRTSABRE

A pass of the BLaRTSABRE algorithm is very similar to that of SABRE and TeleSABRE: executable gates are collected until no more gates on adjacent qubits; then the algorithm collects possible candidate operations, evaluates their score using a heuristic, then executes the best operation. The biggest simplifications come from the candidate selection subroutine and heuristic function. The crux of these two improvements come from how the BLaRT edge is represented in the algorithm architecture coupling graph G_{BLaRT}. By abstracting each BLaRT edge as a bipartite connection (Fig. 11), the connectivity of a BLaRT edge can be properly represented and utilised in the algorithm.

Instead of collecting candidates across Local SWAPs, Teleportations, and Telegates, BLaRTSABRE only collects SWAPs. Like in SABRE, any edge incident to a qubit of a gate in $\mathcal{F}$ is collected as a candidate SWAP. By treating the abstracted BLaRT edges identically to intra-core edges, the algorithm is able to collect both Local SWAPs and Remote SWAPs simultaneously. To determine if a candidate SWAP is local or remote, it is sufficient to check the coupling type of the edge it acts over, as either `Local` or `BLaRT`.

Similarly, Remote Gates are not collected as Telegates would be; when the algorithm checks if the architecture coupling supports the execution of a gate, BLaRT edges are allowed to act as a coupling edge. Thus, an operation is automatically detected as a Remote Gate if the coupling type of the edge it acts over is `BLaRT`, rather than `Local`.

As BLaRT edges act uniformly to normal coupling edges, BLaRTSABRE uses SABRE's unmodified heuristic function:

$$H(\sigma) = \delta(\sigma)\left(\frac{1}{|\mathcal{F}|}\sum_{g\in\mathcal{F}} D(\pi_\sigma(g[l_1]),\pi_\sigma(g[l_2])) + W\cdot\left(\frac{1}{|\mathcal{E}|}\sum_{g\in\mathcal{E}} D(\pi_\sigma(g[l_1]),\pi_\sigma(g[l_2]))\right)\right) \quad (1)$$

Where D is the distance function on the abstracted BLaRT graph G_{BLaRT}. However, the fact that BLaRT edges have a greater overhead than local operations and increase circuit depth must be taken into account. Therefore, all abstracted BLaRT edges are assigned a greater weight (around 3-10) than the default weight of 1 that intra-core coupling edges have.

Additionally, the following catalogues all the TeleSABRE optimisations that should be applied in a BLaRTSABRE implementation:

- **Deadlock & Reset Timer:** Deadlocks have been prevented, but in case of the "ping-pong" effect, the deadlock solving and reset timers have remained in the lesser chance that conflicting dependencies prevent the algorithm from progressing.
- **Random Best Operation:** Remains in place to prevent "ping-pong"ing.
- **Initial Mapping Optimisation:** Initial mapping optimisations still ensures the first couple of gates do not require the overhead of Remote Teleportations, so this optimisation stays in use.
- **Iterations:** As the algorithm results are still random, performing multiple iterations helps produce a better result.

Our GitHub repository of BLaRTSABRE (and a customised implementation of SABRE and TeleSABRE) is accessible at [9].

Algorithm 2 BLaRTSABRE Forward Pass

Input: Circuit DAG, Architecture G_{BLaRT}
while FrontLayer is not empty **do**
    $execute\_gate\_list \leftarrow [\,]$
    $gate\_execution\_log \leftarrow [\,]$
    $decay \leftarrow$ OnesArray($G_{BLaRT}.num\_qubits$)
    for *gate in* FrontLayer **do**
        if $G_{BLaRT}.is\_gate\_executable(\pi(gate))$ **then**
            $execute\_gate\_list.append(gate)$
        end if
    end for
    if $execute\_gate\_list = [\,]$ **then**
        for *gate in* $execute\_gate\_list$ **do**
            DAG.$remove(gate)$
            if $gate.coupling\_edge\_type =$ `BLaRT` **then**
                $gate\_execution\_log.append(gate,$ `remote`$)$
            else
                $gate\_execution\_log.append(gate,$ `local`$)$
            end if
        end for
        $solving\_deadlock \leftarrow False$
        $solving\_deadlock\_timer \leftarrow 80$
        $reset\_timer \leftarrow 150$
    else
        Candidates $C \leftarrow$ GetSwaps($\mathcal{F}$)
        $score \leftarrow [\,]$
        for σ in C **do**
            $\pi_\sigma \leftarrow$ApplyOperation(π, σ)
            $score[\sigma] \leftarrow$ CalculateScore($\sigma, \mathcal{F}, \mathcal{E}, decay, solving\_deadlock$)
        end for
        $best\_\sigma \leftarrow \min(score)$
        $\pi \leftarrow$ApplyOperation($\pi, best\_\sigma$)
        $decay \leftarrow$ UpdateDecay($decay, best\_\sigma$)
        if $best\_\sigma.SWAP\_edge\_type =$ `BLaRT` **then**
            $gate\_execution\_log.append(best\_\sigma,$ `remote`$)$
        else
            $gate\_execution\_log.append(best\_\sigma,$ `local`$)$
        end if
    end if
    Update DAG, $\mathcal{F}, \mathcal{E}$
    UpdateTimers($solving\_deadlock, solving\_deadlock\_timer, reset\_timer$)
end while
return $\pi, gate\_execution\_log$

6 Experimental Results

In this section, we performed benchmarking simulations to compare our algorithm implementations against the current state-of-the-art in order to test the effectiveness of our algorithms. Evaluation was performed on a limited series of standard QASM quantum circuit benchmarks on two sizes of architecture: "Small" circuits (25 qubits) that were executed on a 4-core architecture (Figs. 2 and 8), and "Large" circuits (64 qubits) that were executed on a 6-core architecture. Each circuit was run with 1-5 iterations and the performance statistics of the best iteration was used in our analysis. Well-performing algorithms were further tested using the quekno circuit benchmarking dataset [10], providing a greater selection of small and large complex circuits. The benchmarking programs and algorithm implementations may be found in our repository [9].

6.1 Our SABRE Implementation vs. Qiskit LightSABRE Implementation

As a baseline test, our SABRE implementation was compared to the current routing standard, Qiskit's LightSABRE [2]. LightSABRE has three variants of its heuristic function: Basic, Lookahead, and Decay. Our implementation shows a clear improvement over the Basic heuristic, but it has not demonstrated a competitive improvement over Lookahead and Decay when benchmarked with the quekno dataset. The scatter plot in Figure 16 provides an overview of our SABRE's performance compared to the three LightSABRE variants. Basic LightSABRE appears to spread out towards the upper-right portion of the scatter plot, providing clear evidence for it as the weakest variant. In comparison, the Lookahead and Decay LightSABRE simulations tend to cluster very similarly. Our SABRE's data points tend to match these clusters, but are noticeably shifted to the right, indicating higher SWAP operation counts. Moreover, the distributions of SWAP count and circuit depth in Figure 16 demonstrate that the Lookahead and Decay variants perform near-identically, in comparison to the greater variance present in our SABRE implementation. Even though the averages in Figure 16 are skewed by the presence of large circuits in the dataset, it illustrates that our average SABRE SWAP count falls between variants of Lookahead/Decay and Basic. However, the average depth of our SABRE implementation is slightly higher than all three LightSABRE variants. This suggests that, even without LightSABRE's optimisations, a typical SABRE implementation can perform on par with (or slightly under) the current industry standard for routing on *monolithic* devices.

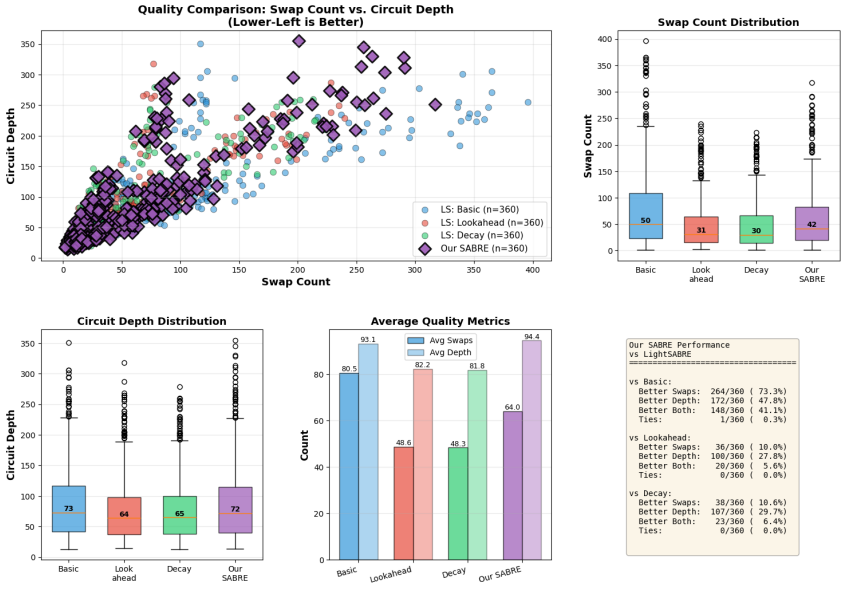
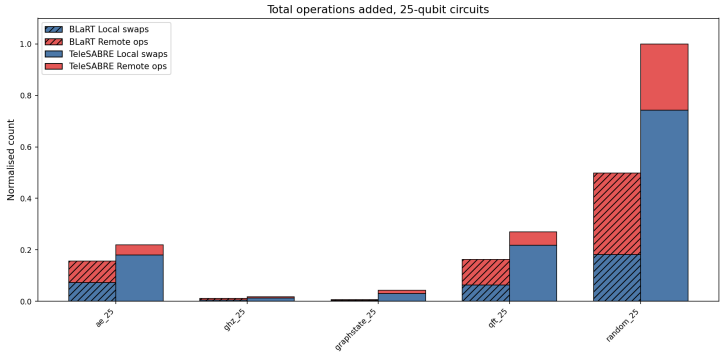


Figure 16: Comparison of Our SABRE against LightSABRE Variations

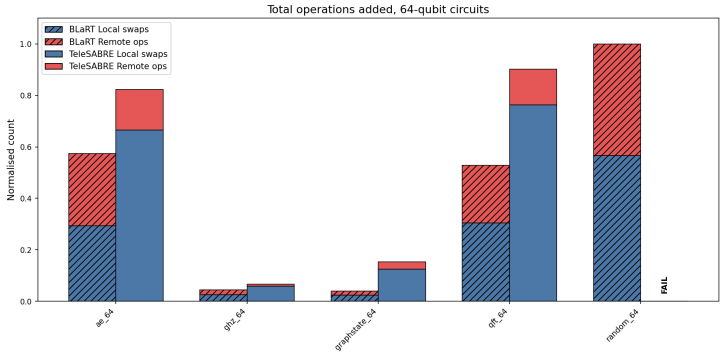
6.2 BLaRTSABRE vs. TeleSABRE Implementation

BLaRTSABRE consistently inserts fewer total operations than TeleSABRE, as illustrated in Figure 17. The BLaRTSABRE advantange grows with circuit size, where we can observe a modest gap in total operations inserted into 25-qubit circuits (Fig. 17a) to a significant jump in the difference of total operations inserted into 64-qubit circuits (Fig. 17b) after a successful routing by each algorithm.

In addition, BLaRTSABRE's deadlock-prevention measures have been demonstrated to be effective, as it was able to route a large 64-qubit circuit with 5400 gates that failed on TeleSABRE due to an unsolvable deadlock. The longer and more complex a circuit is, the more likely it is for a Teleportation to disrupt the balance of the distributed free qubits and create full cores, which is not a problem faced by Remote SWAPs.



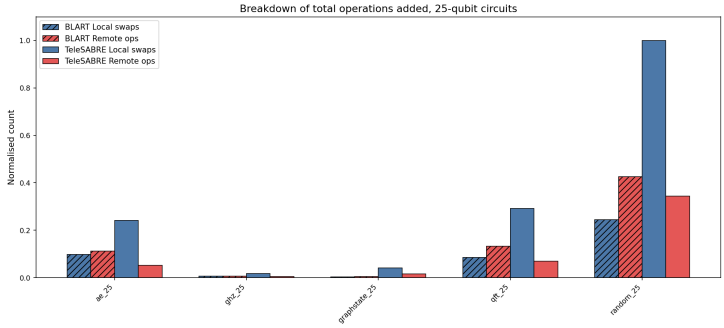
(a) 25-qubit circuits



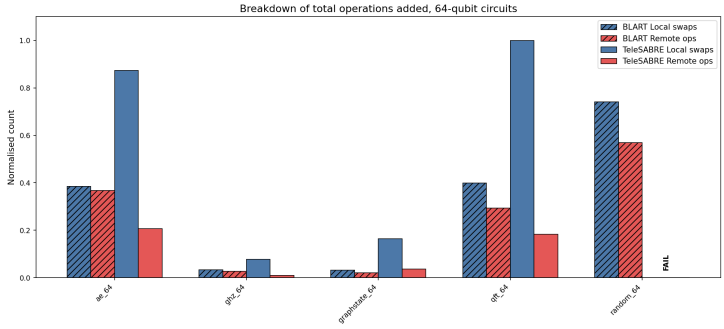
(b) 64-qubit circuits

Figure 17: Comparison of total operations added after routing of BLaRTSABRE and TeleSABRE

However, BLaRTSABRE tends to favour Remote Teleportation operations over Local SWAPs, as shown when we break it down by Remote and Local operations. As seen in Figure 18, TeleSABRE circuits have over twice as many Local SWAPs as BLaRTSABRE. Since inter-core operations such as Teleportations, Telegates, and Remote SWAPs each require the overhead of Bell-Linking, cat-entanglement, and cat-detanglement, then Local SWAPs are much cheaper than inter-core operations. However, a clear counterargument can be made from Figure 17, where the total number of inserted operations from a successful routing of TeleSABRE is consistently significantly higher than BLaRTSABRE. From this, we can deduce that the number of saved Local SWAPs offsets the increase in Remote Teleportations, and as such, depth of the circuit routed by BLaRTSABRE will still be significantly lower than TeleSABRE's. In the era of NISQ, where system coherence is limited by time, a significant decrease in depth means that circuits that cannot be executed due to the TeleSABRE compilation method may be executable with BLaRTSABRE compilation.



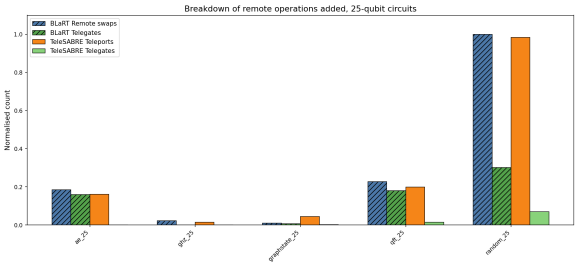
(a) 25-qubit circuits



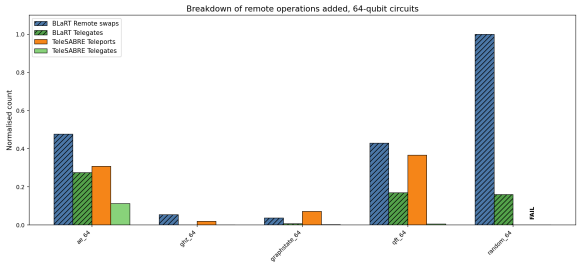
(b) 64-qubit circuits

Figure 18: Breakdown of total operations added after routing of BLaRTSABRE and TeleSABRE

Furthermore, we can further break down the inter-core operations inserted by TeleSABRE and BLaRTSABRE, as shown in Figure 19. We observe that Remote SWAPs and Teleportations occur nearly equally in the 25-qubit circuits (Fig. 19a). Although, it is pertinent to remember that Remote SWAPs involve twice as many operations as Teleportation (though without increasing depth). In comparison, we can also observe that BLaRTSABRE uses many more Remote Gates than TeleSABRE uses Telegates in both 25-qubit and 64-qubit circuits. However, with careful circuit compilation, many Remote gates can be executed in parallel to take advantage of BLaRT edges' double capacity, reducing the depth of Remote Gates by up to half (further discussed in Section 7.2). This reduces the burden of Remote SWAP costs.



(a) 25-qubit circuits



(b) 64-qubit circuits

Figure 19: Breakdown of remote operations added after routing of BLaRTSABRE and TeleSABRE

As an additional bonus, the BLaRTSABRE algorithm executes much faster than the TeleSABRE implementation, saving an average of 30 seconds per iteration (up to 3 minutes faster in the best case, and 1 minute slower in the worst case).

Overall, BLaRTSABRE demonstrates a clear improvement in quantum circuit compilation with only a single modification to the DQC architecture.

7 Further Discussion

7.1 The Contribution of BLaRTSABRE

As demonstrated in Section 6.2, BLaRTSABRE demonstrates itself as a strong contender for the current standard in DQC compilation. In comparison to TeleSABRE: it is faster, it inserts fewer total operations, its inter-core operations have a lower depth, it is significantly less prone to getting affected by the "ping-pong" effect, it can handle larger and more complex circuits, and it cannot get stuck in a deadlock. The most notable downside that BLaRTSABRE has when compared to TeleSABRE is that it requires a specialised architecture. TeleSABRE can be executed on any modular architecture as long as the coupling edges can entangle a Bell state and send control bit data between cores, whereas BLaRTSABRE requires BLaRT architecture, requiring *every* coupling edge between cores to be a BLaRT edge. As each BLaRT edge is comprised of 6 entire qubits and many connecting edges, the physical engineering of a BLaRT edge is the most consequential bottleneck of the entire system. While the qubits within a BLaRT node perform a limited set of predictable, well-defined operations, it is the nature of NISQ computing that every additional qubit multiplies the system's complexity and adds another vector for decoherence. As such, we expect that BLaRTSABRE may only be practically applicable on larger, late-NISQ architectures with greater qubit capacity management.

7.2 Future Work

While BLaRTSABRE effectively manages the trade-off between local and remote operations, several avenues for optimization remain:

- **Burst Communication:** Inter-core operations, particularly Telegates and Remote Gates, benefit from the application of Burst Communication, as described in the AutoComm framework [3]. Burst Communication takes advantage of the nature of Telegates, where it is possible to execute multiple gates between two cores using the same control qubit, all in a single Tele-operation. By implementing the AutoComm framework into a circuit compilation pipeline, Remote Gates could significantly cut down their depth and cost, executing multiple Telegates simultaneously using just one of the two available communication edges in a BLaRT edge.

- **Parallelised Compilation:** The current implementation of BLaRTSABRE performs each stochastic search iteration sequentially. Since each routing iteration is an independent routine, the algorithm can be trivially parallelised across multiple CPU threads. This would allow for concurrent exploration of the search space, significantly reducing the wall-clock time required to find an optimal mapping.

- **Code Refactoring:** The majority of the programming in this investigation was done in Python, due to its simplicity and flexibility for prototyping algorithms. However, BLaRTSABRE could benefit greatly from being ported to a lower-level, compiled language such as C. This would allow routing operations to be executed in seconds, rather than minutes.

- **Greater Benchmarking Dataset:** The benchmarking simulations in this analysis were only performed on two primary datasets and on two simple grid-type architectures. In order to test the limits of BLaRTSABRE's capabilities, a further investigation is warranted, evaluating BLaRTSABRE on both a larger set of sample circuits with a greater variance in the number of qubits, as well as a larger sample of DQC hardware architectures with novel coupling topologies to test how the algorithm functions in esoteric situations.

8 Conclusion

In this work, we presented BLaRTSABRE, a compilation protocol for multi-core quantum architectures. Through the use of the novel BLaRT architecture, the SABRE heuristic was shown to be applicable in the DQC paradigm; we demonstrated that BLaRTSABRE is a viable and effective approach for mapping quantum circuits onto heterogeneous topologies. Our findings underscore that practical DQC compilation requires not just stochastic distance optimisation, but active management of the scarce resources involved in quantum entanglement on NISQ-era devices. BLaRT architecture is but another abstraction on top of all the other abstractions comprising the entirety of modern quantum computing.

References

[1] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019. DOI: 10.1145/3297858.3304023.

[2] H. Zou, M. Treinish, K. Hartman, A. Ivrii, and J. Lishman, "Lightsabre: A lightweight and enhanced SABRE algorithm," *arXiv preprint arXiv:2409.08368*, 2024. [Online]. Available: https://arxiv.org/abs/2409.08368.

[3] A. Wu, H. Zhang, G. Li, et al., "Autocomm: A framework for enabling efficient communication in distributed quantum programs," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022.

[4] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, "Telesabre: Layout synthesis in multi-core quantum systems with teleport interconnect," *arXiv preprint arXiv:2505.08928*, 2025. [Online]. Available: https://arxiv.org/abs/2505.08928.

[5] S. Bravyi and A. Kitaev, "Universal quantum computation with ideal clifford gates and noisy ancillas," *Phys. Rev. A*, vol. 71, p. 022 316, 2 Feb. 2005. DOI: 10.1103/PhysRevA.71.022316. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevA.71.022316.

[6] S. Hillmich, A. Zulehner, and R. Wille, "Exploiting quantum teleportation in quantum circuit mapping," Jan. 2021, pp. 792–797. DOI: 10.1145/3394885.3431604.

[7] A. Yimsiriwattana and S. J. L. Jr, *Generalized ghz states and distributed quantum computing*, 2004. arXiv: quant-ph/0402148 [quant-ph]. [Online]. Available: https://arxiv.org/abs/quant-ph/0402148.

[8] E. Russo, *Github - haimrich/telesabre: Telesabre: Layout synthesis in multi-core quantum systems with teleport interconnect*, 2025. [Online]. Available: https://github.com/Haimrich/telesabre.

[9] E. Blaustein, S. Ravindran, and T. P. Tran, *Github - sibishan/blartsabre*, 2026. [Online]. Available: https://github.com/sibishan/blartsabre.

[10] S. Li, *Github - ebony72/quekno: Quekno: A benchmark construction algorithm for quantum circuit transformation*, 2024. [Online]. Available: https://github.com/ebony72/quekno.